COMPILED BY

MANISH KUMAR YADAV

PLAYWRIGHT INTERVIEW QUESTIONS

Complete Guide with Detailed Answers
Basic & Advanced Level

45 QUESTIONS COVERED

- Basic Questions (1-25)
- Advanced Questions (1-20)
- Code Examples & Best Practices
- Configuration & Setup Guides

2026 EDITION

PLAYWRIGHT INTERVIEW QUESTIONS & ANSWERS

BASIC QUESTIONS (1-25)

Q1. What is Playwright?

Playwright is an open-source browser automation library developed by Microsoft. It is designed for testing modern web applications and supports Chromium, Firefox, and WebKit browsers through a single unified API. Its main goal is to provide reliable, fast, and capable browser automation that can handle complex scenarios of modern web apps.

Q2. What are the advantages of Playwright?

- **Cross-browser support**: Chromium, Firefox, and WebKit via single API
- **Auto-waiting**: Built-in intelligent waiting mechanisms
- **Speed**: Faster execution compared to Selenium
- **Multi-language support**: JavaScript, TypeScript, Python, Java, .NET
- **Headless & Headed modes**: Both modes supported
- **Built-in test runner**: @playwright/test package included
- **Parallel execution**: Through workers
- **Tracing & Debugging**: Built-in trace viewer, screenshots, videos
- **Mobile emulation**: Device viewport simulation
- **Network interception**: API mocking and request/response handling

Q3. What are the disadvantages of Playwright?

- Relatively new compared to Selenium, so community and resources are fewer
- Learning curve exists for advanced features
- No Internet Explorer support
- Resource intensive when running multiple browsers
- Limited legacy system integration can be difficult

Q4. Which programming languages are supported by Playwright?

Playwright supports: JavaScript, TypeScript (recommended), Python, Java, and .NET/C#.

Q5. What types of tests does Playwright support?

- End-to-End (E2E) Testing
- Functional Testing
- API Testing (HTTP requests)
- Visual Regression Testing (screenshot comparison)
- Accessibility Testing (axe-core integration)
- Component Testing (experimental)
- Performance Testing (basic level)

Q6. How do Selenium and Playwright differ?

Architecture: Selenium uses WebDriver protocol while Playwright uses native DevTools protocols.
Waiting: Selenium requires manual explicit/implicit waits; Playwright has built-in auto-waiting.
Speed: Playwright is faster due to direct browser communication.
Parallel Execution: Playwright has built-in workers; Selenium requires Grid setup.
Mobile: Playwright offers better device emulation.
Tracing: Playwright has built-in trace viewer; Selenium uses external tools.

Q7. How to setup & install Playwright?

Run these commands:

```
npm init -y
npm init playwright@latest
npx playwright install
npx playwright --version
```

Q8. How to setup a new Playwright project?

Use automatic setup: **npm init playwright@latest my-project**. This creates playwright.config.ts, tests/ folder, tests-examples/ folder, and updates package.json.

Manual config example:

```
import { defineConfig, devices } from '@playwright/test';
export default defineConfig({
testDir: './tests',
fullyParallel: true,
reporter: 'html',
use: { baseURL: 'http://localhost:3000', trace: 'on-first-retry' },
projects: [
{ name: 'chromium', use: { ...devices['Desktop Chrome'] } },
{ name: 'firefox', use: { ...devices['Desktop Firefox'] } },
{ name: 'webkit', use: { ...devices['Desktop Safari'] } },
],
});
```

Q9. Does Playwright require a Webdriver dependency?

No, Playwright does not require WebDriver. It uses native DevTools protocols directly (CDP for Chromium, Juggler for Firefox, WebKit protocol), making it faster and more reliable than WebDriver-based tools.

Q10. Can you describe Playwright's architecture?

Playwright's architecture has three layers:
1. **Language Bindings Layer**: JS/TS, Python, Java, C# APIs
2. **Playwright Library**: Core logic, auto-waiting, selectors
3. **Browser Drivers**: Chromium (CDP), Firefox (Juggler), WebKit (custom protocol)

Communication flows: Language bindings ↔ Playwright Server ↔ Browser instances via WebSocket-like connections.

Q11. How does Playwright differ from other testing frameworks?

vs Cypress: Playwright supports multi-tab, multi-origin, and cross-browser better. Cypress is limited to single-tab.
vs Selenium: Playwright uses auto-waiting and native protocols; Selenium depends on WebDriver.
vs Puppeteer: Playwright is cross-browser (Puppeteer is Chromium-only) with more testing-specific features.
vs TestCafe: Playwright provides better debugging and tracing capabilities.

Q12. What is a configuration file in Playwright?

A configuration file defines test behavior including browsers, timeouts, reporters, base URLs, and other global settings.

Q13. What is the typical configuration file name in Playwright?

- playwright.config.ts (TypeScript - recommended)
- playwright.config.js (JavaScript)
- playwright.config.mjs (ES Module)

Q14. What is the @playwright/test Package?

It is Playwright's official test runner with built-in features: test organization (test, expect), parallel execution, fixtures, reporters (HTML, JSON, JUnit), hooks (beforeEach, afterEach, beforeAll, afterAll), and annotations (test.skip, test.only, test.fixme).

Q15. What is the Page Class in Playwright?

Page class represents a browser tab or window. Through it you can navigate (page.goto()), interact with elements (page.click(), page.fill()), take screenshots, and listen to events.

```
const page = await browser.newPage();
await page.goto('https://example.com');
await page.click('button#submit');
```

Q16. How to navigate to specific URLs in Playwright?

```
await page.goto('https://example.com');
await page.goto('https://example.com', { waitUntil: 'networkidle', timeout: 30000 });
await page.goto('/dashboard'); // if baseURL is set
```

Q17. What types of reporters does Playwright support?

Built-in: list (default), line, dot, html, json, junit, github. Third-party: Allure, Monocart, etc.

```
reporter: [
['html', { open: 'never' }],
['json', { outputFile: 'results.json' }],
['junit', { outputFile: 'results.xml' }]
],
```

Q18. What are Locators in Playwright? Name some Locators.

Locators are robust ways to find elements with auto-waiting and retry logic.

Types:
- page.getByRole('button', { name: 'Submit' })
- page.getByText('Welcome') / page.getByText(/welcome/i)
- page.getByLabel('Password')
- page.getByPlaceholder('Enter email')
- page.getByAltText('Company Logo')
- page.getByTitle('Close')
- page.getByTestId('login-button')
- page.locator('.button.primary') // CSS
- page.locator('xpath=//button[@id="submit"]') // XPath

Q19. What types of text selectors does Playwright offer?

- Exact text: page.getByText('Exact Text')
- Substring: page.locator('text=Partial Text')
- Regex: page.getByText(/regex pattern/i)
- Has-text pseudo-class: page.locator('article:has-text("News")')

Q20. List out some assertions in Playwright.

```
import { expect } from '@playwright/test';

// Element assertions
await expect(page.locator('h1')).toBeVisible();
await expect(page.locator('h1')).toBeHidden();
await expect(page.locator('h1')).toHaveText('Welcome');
await expect(page.locator('h1')).toContainText('Wel');
await expect(page.locator('input')).toHaveValue('test');
await expect(page.locator('button')).toBeEnabled();
await expect(page.locator('button')).toBeDisabled();
await expect(page.locator('img')).toHaveAttribute('src', /logo/);
await expect(page.locator('div')).toHaveClass('active');
await expect(page.locator('div')).toHaveCount(5);

// Page assertions
await expect(page).toHaveTitle('Home Page');
await expect(page).toHaveURL(/dashboard/);

// Screenshot assertions
await expect(page).toHaveScreenshot('landing.png');
```

Q21. How to use assertions in Playwright?

```
import { test, expect } from '@playwright/test';

test('login works', async ({ page }) => {
await page.goto('/login');
await page.getByLabel('Email').fill('user@example.com');
await page.getByLabel('Password').fill('password123');
await page.getByRole('button', { name: 'Login' }).click();
await expect(page).toHaveURL('/dashboard');
await expect(page.getByText('Welcome back')).toBeVisible();
await expect(page.locator('.error')).not.toBeVisible();
});
```

Q22. What are hard assertions and soft assertions in Playwright?

Hard Assertions (Default): Test stops immediately on failure. Uses expect().
Soft Assertions: Test continues on failure, fails at end. Uses expect.soft(). Useful for checking multiple conditions at once.

```
test('soft assertion example', async ({ page }) => {
await expect.soft(page.locator('h1')).toHaveText('Wrong');
await expect.soft(page.locator('h2')).toHaveText('Also Wrong');
await expect.soft(page.locator('p')).toBeVisible();
});
```

Q23. Does Playwright support XPath? How to implement it?

Yes, but not recommended. Priority order: role > text > CSS > XPath.

```
await page.locator('xpath=//button[@id="submit"]').click();
await page.locator('//div[contains(@class, "active")]').click();
await page.locator('css=.container >> xpath=.//button').click();
```

Q24. Does Playwright support HTML reporters? How to generate?

Yes, Playwright has built-in HTML reporter with trace viewer, screenshots, and videos.

```
reporter: [['html', { outputFolder: 'playwright-report', open: 'on-failure' }]],
```

Commands:
npx playwright test --reporter=html
npx playwright show-report

Q25. What is the difference between Playwright and Selenium?

Protocol: Playwright uses Native DevTools; Selenium uses WebDriver.
Speed: Playwright is faster with direct communication.
Waiting: Playwright has auto-waiting; Selenium needs manual waits.
Parallel: Playwright has built-in workers; Selenium needs Grid.
Mobile: Playwright has native emulation; Selenium needs Appium.
Tracing: Playwright has built-in trace viewer; Selenium uses external tools.
Flakiness: Playwright is less flaky due to better timing.
Community: Selenium has larger community; Playwright is growing rapidly.

ADVANCED QUESTIONS (1-20)

Q1. What common challenges have you encountered with Playwright?

Flaky Tests: Due to dynamic content. Solution: Use proper waiting strategies, data-testid attributes.
Third-party Iframes: Solution: Use page.frameLocator().
File Uploads: Solution: Use page.setInputFiles() with correct paths.
Authentication: Solution: Save storageState in config or use global setup.
Parallel Data Conflicts: Solution: Generate unique test data, use isolated accounts.

Q2. How to wait for a specific element in Playwright?

```
// Auto-waiting (Recommended)
await page.locator('.loading').waitFor({ state: 'hidden' });
await page.getByRole('button').click(); // Auto-waits

// Explicit wait
await page.waitForSelector('.item', { timeout: 5000 });

// Wait for state
await page.locator('.spinner').waitFor({ state: 'detached' });

// Wait for function
await page.waitForFunction(() => document.querySelector('.item') !== null);

// Wait for response
await page.waitForResponse('**/api/data');
```

Q3. Can you run tests in parallel in Playwright?

Yes, Playwright supports built-in parallel execution through workers.

```
export default defineConfig({
fullyParallel: true,
workers: 4, // or undefined for auto
});

// Parallel within file
test.describe.configure({ mode: 'parallel' });
```

Q4. In what ways does Playwright manage browser automation?

- **Browser Contexts**: Lightweight isolated sessions
- **Pages**: Tabs within context
- **Auto-waiting**: Waits before actions
- **Actionability checks**: Element must be visible, enabled, stable
- **Retry logic**: Failed actions retry automatically
- **Tracing**: Records every step for debugging
- **Fixtures**: Reusable setup/teardown logic

Q5. How does Playwright handle asynchronous operations? Give an example.

Playwright is Promise-based and uses async/await.

```
test('async operations', async ({ page }) => {
await page.goto('https://example.com');

const [response] = await Promise.all([
page.waitForResponse('**/api/users'),
page.click('button#load-users')
]);

expect(response.status()).toBe(200);

await Promise.all([
page.click('button-1'),
page.click('button-2'),
page.waitForSelector('.result')
]);
});
```

Q6. What are the best methods for debugging tests in Playwright?

1. **Trace Viewer** (Best): npx playwright test --trace on
2. **Headed Mode**: npx playwright test --headed
3. **PWDEBUG**: PWDEBUG=1 npx playwright test (step-by-step)
4. **Slow Motion**: use: { launchOptions: { slowMo: 500 } }
5. **Screenshots & Videos**: use: { screenshot: 'only-on-failure', video: 'retain-on-failure' }
6. **VS Code Extension**: Official Playwright extension with breakpoints

Q7. Best practices for writing efficient and maintainable tests?

1. **Page Object Model (POM)**: Encapsulate page logic
2. **Fixtures**: Reusable setup
3. **data-testid**: Avoid fragile selectors
4. **Independent tests**: No test dependencies
5. **Global setup**: For authentication
6. **Meaningful names**: Clear test descriptions
7. **Tags**: @smoke, @regression
8. **Soft assertions**: Where appropriate

```
class LoginPage {
constructor(private page: Page) {}
async login(email: string, password: string) {
await this.page.getByLabel('Email').fill(email);
await this.page.getByLabel('Password').fill(password);
await this.page.getByRole('button', { name: 'Login' }).click();
}
}
```

Q8. Best practices for using locators effectively?

Priority Order (Best to Worst):

1. getByRole() - Accessibility based (most resilient)
2. getByText() - User-visible text
3. getByLabel() - Form labels
4. getByPlaceholder() - Input placeholders
5. getByAltText() - Images
6. getByTitle() - Title attributes
7. getByTestId() - Custom data attributes
8. CSS selectors - Last resort
9. XPath - Avoid if possible

```
// Good
await page.getByRole('button', { name: 'Submit order' }).click();

// Bad
await page.locator('div > button.btn-primary:nth-child(2)').click();
```

Q9. What common errors and troubleshooting methods?

TimeoutError: Check locator, increase wait
StrictModeViolation: Use .first(), .nth(0), or more specific locator
Target closed: Check memory, try headless mode
Navigation failed: Verify URL, check network trace
Execution context destroyed: Retry action or add wait

Browser not found: Run npx playwright install

Q10. What are Headed and Headless modes?

Headless (Default): No UI shown, faster execution, ideal for CI/CD, no GUI overhead.
Headed: Browser UI visible, useful for debugging, visual verification possible, slightly slower.

Commands:
npx playwright test --headed
use: { headless: true }

Q11. What measures for cross-browser compatibility?

- Unified API for all browsers
- Native protocols for each browser
- Same test code across browsers
- Projects configuration for different browsers
- Consistent auto-waiting and actionability checks
- Cross-browser visual regression testing

```
projects: [
{ name: 'chromium', use: { ...devices['Desktop Chrome'] } },
{ name: 'firefox', use: { ...devices['Desktop Firefox'] } },
{ name: 'webkit', use: { ...devices['Desktop Safari'] } },
{ name: 'Mobile Chrome', use: { ...devices['Pixel 5'] } },
{ name: 'Mobile Safari', use: { ...devices['iPhone 12'] } },
],
```

Q12. How does Playwright integrate with CI tools?

Playwright provides official CI guides for GitHub Actions, Azure Pipelines, CircleCI, Jenkins, GitLab CI, and Docker support.

```
# GitHub Actions example
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
- run: npm ci
- run: npx playwright install --with-deps
- run: npx playwright test
- uses: actions/upload-artifact@v4
if: failure()
with:
name: playwright-report
path: playwright-report/
```

Q13. Does Playwright include built-in reporting?

Yes. Built-in reporters: list, line, dot, html, json, junit. Third-party: Allure, Monocart.

```
reporter: [
['html'],
['json', { outputFile: 'test-results.json' }],
['junit', { outputFile: 'test-results.xml' }]
],
```

Q14. What are timeouts in Playwright?

Timeouts specify how long an operation waits before failing.

Q15. What are the different types of timeouts?

test timeout: 30s default - max time for entire test
expect timeout: 5s default - assertion wait time
action timeout: 0 (no limit) - for click, fill, etc.
navigation timeout: 0 (no limit) - for goto, reload

```
export default defineConfig({
timeout: 30000,
expect: { timeout: 5000 },
use: {
actionTimeout: 0,
navigationTimeout: 0
},
});

// Action-specific
await page.click('button', { timeout: 10000 });
```

Q16. How to navigate forward and backward?

```
// Backward
await page.goBack();

// Forward
await page.goForward();
```

```
// With options
await page.goBack({ waitUntil: 'networkidle', timeout: 10000 });

// Refresh
await page.reload();

// Check history
console.log(await page.evaluate(() => history.length));
```

Q17. How to perform actions using Playwright?

```
// Click actions
await page.getByRole('button').click();
await page.getByRole('button').dblclick();
await page.getByRole('button').click({ button: 'right' });
await page.getByRole('button').hover();

// Input actions
await page.getByLabel('Name').fill('John Doe');
await page.getByLabel('Name').clear();
await page.getByLabel('Name').press('Enter');

// Select dropdown
await page.getByLabel('Country').selectOption('India');
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByLabel('Country').selectOption({ value: 'IN' });
await page.getByLabel('Country').selectOption({ index: 2 });

// Checkboxes
await page.getByLabel('I agree').check();
await page.getByLabel('I agree').uncheck();

// File upload
await page.getByLabel('Upload').setInputFiles('path/to/file.pdf');

// Drag and drop
await page.locator('.item').dragTo(page.locator('.drop-zone'));

// Scroll
await page.locator('.section').scrollIntoViewIfNeeded();
```

Q18. Does Playwright support Safari browser?

Yes, Playwright supports WebKit browser which is based on Safari's rendering engine. It is not actual Safari app but uses the same WebKit engine, providing equivalent testing capabilities.

```
npx playwright test --project=webkit

projects: [
{ name: 'webkit', use: { ...devices['Desktop Safari'] } },
],
```

Q19. How to execute Playwright tests on Safari?

Commands:
npx playwright test --project=webkit
npx playwright test login.spec.ts --project=webkit
npx playwright test --project=webkit --headed

```
Config:
projects: [
{ name: 'safari-desktop', use: { ...devices['Desktop Safari'] } },
{ name: 'safari-mobile', use: { ...devices['iPhone 14'] } },
],
```

Q20. Can you use Playwright to scrape dynamic websites?

Yes, Playwright is excellent for scraping dynamic websites because it runs a real browser with JavaScript execution.

```
import { chromium } from '@playwright/test';

async function scrapeDynamicSite() {
const browser = await chromium.launch();
const page = await browser.newPage();

await page.goto('https://dynamic-site.com');
await page.waitForSelector('.loaded-content');

const data = await page.evaluate(() => {
return Array.from(document.querySelectorAll('.item')).map(item => ({
title: item.querySelector('h2')?.innerText,
price: item.querySelector('.price')?.innerText,
}));
});

console.log(data);
await browser.close();
}
```

THANK YOU

For Reading This Guide

Compiled & Prepared by

MANISH KUMAR YADAV

Playwright Interview Questions Guide

2026 Edition | Complete Reference

Best of luck for your interviews!

THANK YOU

For Reading This Guide

Compiled & Prepared by

MANISH KUMAR YADAV

Playwright Interview Questions Guide
2026 Edition | Complete Reference

Best of luck for your interviews!